

Documentacion Final - Proyecto 3

VeriLang en Rascal: parser, generador y sistema de tipos con TypePal

Campo	Informacion
Curso	ISIS-2111 - Elementos Esenciales de Lenguajes de Programacion
Estudiante	Ronny Esteban Lopez Martinez
Codigo	202415741
Proyecto Rascal	rascaldslverilang
Contenido de la entrega	Proyecto Rascal comprimido en ZIP y documento explicativo final

Resumen ejecutivo

Este documento presenta la version final de VeriLang para el Proyecto 3. La solucion implementa un lenguaje definido en Rascal con parser, AST, generador, integracion con TypePal, anotaciones de tipos, verificacion semantica y casos de prueba validos e invalidos. El proyecto toma programas VeriLang desde archivos .vl, los parsea, revisa nombres y tipos, y genera una representacion textual en consola cuando el programa es correcto. Si se encuentran errores semanticos, el flujo principal reporta los errores y detiene la generacion.

Tabla de contenido

1. Introduccion y objetivo
2. Requisitos abordados por la solucion
3. Estructura del proyecto
4. Flujo de ejecucion
5. Gramatica concreta y anotaciones de tipo
6. Sintaxis abstracta e Implode
7. Parser del lenguaje
8. Generador de codigo
9. Sistema de tipos con TypePal
10. Regla de existencia de estructuras y tipos de usuario
11. Integracion en Main.rsc y Plugin.rsc
12. Casos de prueba
13. Como ejecutar el proyecto
14. Fuente de la solucion
15. Conclusiones

1. Introduccion y objetivo

El objetivo del Proyecto 3 es completar la definicion de VeriLang usando Rascal. A partir de la gramatica y la representacion abstracta del lenguaje, la solucion incorpora un parser ejecutable, un generador textual y un sistema de tipos basado en TypePal. El lenguaje permite definir espacios o estructuras de datos, variables, operadores, reglas y expresiones logicas con cuantificadores.

La version final reconoce tipos primitivos y tipos definidos por el usuario. En particular, las estructuras de datos pueden declarar explicitamente el tipo de los elementos que contienen. Por ejemplo, la declaracion `defspace Group : Person end` indica que `Group` es una estructura cuyos elementos son de tipo `Person`.

2. Requisitos abordados por la solucion

Aspecto	Implementacion
Parser del lenguaje	Parser.rsc expone funciones <code>parseMainModule</code> para leer programas VeriLang desde texto o desde una ubicacion loc.
Generacion desde .vl	Main.rsc recibe un archivo .vl, ejecuta el parser, valida el programa y muestra el codigo generado en consola.
Integracion de TypePal	Checker.rsc extiende <code>analysis::typepal::TypePal</code> y el proyecto declara <code>lib://typepal</code> en RASCAL.MF.
Anotaciones de tipo	Domain soporta <code>int</code> , <code>bool</code> , <code>real</code> , <code>string</code> , <code>char</code> y tipos definidos por usuario mediante ID.
Estructuras tipadas	Space acepta <code>defspace Nombre : Tipo end</code> y <code>defspace Sub < Super : Tipo end</code> .
Correspondencia de tipos	Checker.rsc valida relaciones, operadores logicos, operadores infijos personalizados, reglas e invocaciones.
Existencia de elementos/tipos	Los tipos de usuario usados en variables, operadores y estructuras deben existir como <code>defspace</code> .

3. Estructura del proyecto

La solucion se organiza en archivos fuente Rascal, ejemplos de entrada, casos invalidos y salidas generadas como evidencia auxiliar.

```

rascaldslverilang/
  META-INF/
  RASCAL.MF
src/main/rascal/
  Syntax.rsc
  AST.rsc
  Parser.rsc
  Implode.rsc
  Generator.rsc
  Checker.rsc
  Main.rsc
  Plugin.rsc
instance/
  spec1.vl
  spec2.vl
  spec3.vl
  spec_types.vl
  spec_typed_space_good.vl
  spec_typed_space_quantifier_good.vl
errors/

```

```

spec_type_error.vl
spec_unknown_space_error.vl
spec_logic_error.vl
spec_operator_error.vl
spec_rule_error.vl
spec_typed_space_unknown_element_error.vl
spec_typed_space_quantifier_type_error.vl
output/
testGenerator.vl
testGenerator2.vl
pom.xml

```

Archivo	Funcion
Syntax.rsc	Define la gramatica concreta: modulos, imports, espacios, variables, operadores, reglas, expresiones, tipos y literales.
AST.rsc	Define la sintaxis abstracta usada por el generador y el checker.
Parser.rsc	Contiene las funciones publicas para parsear programas VeriLang.
Implode.rsc	Convierte el arbol concreto del parser en el AST.
Generator.rsc	Genera texto VeriLang a partir del AST.
Checker.rsc	Implementa el sistema de tipos y las reglas semanticas con TypePal.
Main.rsc	Ejecuta el flujo principal desde un archivo .vl y muestra resultados en consola.
Plugin.rsc	Registra el lenguaje en el entorno y permite generar archivos desde el editor.

4. Flujo de ejecucion

El flujo principal esta implementado en Main.rsc y sigue estos pasos:

- Recibir una ubicacion loc de un archivo .vl.
- Parsear el archivo usando Parser.rsc y Syntax.rsc.
- Construir un modelo semantico con Checker.rsc y TypePal.
- Filtrar los mensajes de tipo error.
- Si hay errores, imprimirlos y retornar 1 sin generar codigo.
- Si no hay errores, llamar a Generator.rsc, imprimir el resultado generado y retornar 0.

```

import Main;
main();

// Resultado esperado para un programa valido:
// Parser ejecutado correctamente.
// Checker ejecutado correctamente.
// No se encontraron errores semanticos.
// Ejecucion finalizada correctamente.
// int: 0

```

5. Gramatica concreta y anotaciones de tipo

La gramatica concreta se encuentra en Syntax.rsc. La regla Domain representa los tipos disponibles en el lenguaje:

```

syntax Domain
  = boolDomain: 'bool'
  | intDomain: 'int'
  | realDomain: 'real'
  | stringDomain: 'string'
  | charDomain: 'char'
  | nameDomain: ID domainName
;

```

Los espacios o estructuras de datos admiten una forma simple y una forma tipada. La forma tipada permite declarar el tipo de los elementos contenidos en la estructura:

```

syntax Space
  = space: 'defspace' ID spaceName 'end'
  | typedSpace: 'defspace' ID spaceName ':' Domain elementType 'end'
  | subspace: 'defspace' ID subSpace '<' ID superSpace 'end'
  | typedSubspace: 'defspace' ID subSpace '<' ID superSpace ':' Domain elementType 'end'
;

```

Ejemplo de uso en un programa VeriLang:

```

defspace Person end
defspace Group : Person end

```

La gramatica tambien reconoce operadores relacionales, operadores logicos y cuantificadores. El cuerpo de un cuantificador debe ser una expresion completa despues del punto.

6. Sintaxis abstracta e Implode

AST.rsc mantiene una representacion algebraica alineada con las etiquetas de la gramatica. Esto permite que Implode.rsc use implode de Rascal para convertir automaticamente el arbol concreto en el AST del lenguaje.

```

data Space
  = space(str spaceName)
  | typedSpace(str spaceName, Domain elementType)
  | subspace(str subSpace, str superSpace)
  | typedSubspace(str subSpace, str superSpace, Domain elementType)
;

public MainModule implodeMain(Tree pt) = implode(#MainModule, pt);
public MainModule load(loc l) = implodeMain(parseMainModule(l));

```

7. Parser del lenguaje

Parser.rsc ofrece dos formas de parsear un programa VeriLang: desde una cadena fuente o desde una ubicacion loc. Esto permite probar el lenguaje tanto desde la terminal de Rascal como desde archivos .vl del proyecto.

```

public start[MainModule] parseMainModule(str src, loc origin)
  = parse(#start[MainModule], src, origin);

public start[MainModule] parseMainModule(loc origin)
  = parse(#start[MainModule], origin);

```

8. Generador de código

Generator.rsc recibe un árbol concreto, lo transforma al AST con Implode.rsc y recorre el resultado para volver a producir texto VeriLang. El generador incluye soporte para estructuras tipadas.

```
str generator(Tree cst) {
    MainModule ast = implodeMain(cst);
    return generate(ast);
}

str generate(typedSpace(str spaceName, Domain elementType)) {
    return "defspace <spaceName> : <generate(elementType)> end";
}
```

El resultado generado se muestra en consola desde Main.rsc para cumplir el flujo de ejecución sobre archivos .vl.

9. Sistema de tipos con TypePal

Checker.rsc define los tipos semánticos del lenguaje. Los tipos primitivos se representan directamente; los tipos definidos por usuario se representan como userType(str name); y los operadores se modelan como functionType(list[AType] signature).

```
data AType
= boolType()
| intType()
| realType()
| stringType()
| charType()
| userType(str name)
| functionType(list[AType] signature)
;
```

También se definen roles de identificadores para distinguir variables, operadores y espacios:

```
data IdRole
= variableId()
| operatorId()
| spaceId()
;
```

9.1 Variables y dominios

Cada declaración defvar registra la variable con el tipo indicado. Si el dominio es un tipo de usuario, el checker exige que exista como espacio definido por defspace.

9.2 Operadores y expresiones

Los operadores personalizados tienen una firma completa. Por ejemplo, belongsTo recibe Person y Group, y retorna bool. Al usarlo como infijo, el checker compara el tipo del lado izquierdo y del lado derecho contra la firma declarada.

```
defoperator belongsTo : Person -> Group -> bool end
defexpression p belongsTo g end
```

9.3 Cuantificadores sobre estructuras tipadas

Cuando un cuantificador itera sobre una estructura tipada, la variable ligada toma el tipo de los elementos de esa estructura. Si Group esta declarado como Group : Person, entonces member en forall member in Group tiene tipo Person.

```
defspace Person end
defspace Group : Person end

defoperator belongsTo : Person -> Group -> bool end

defexpression (forall member in Group . member belongsTo g) end
```

Esta informacion se conserva en Checker.rsc mediante un mapa de tipos de elementos:

```
map[str, AType] typedSpaceElementTypes = ();

AType quantifiedVariableType(str domainName) {
  if (domainName in typedSpaceElementTypes) {
    return typedSpaceElementTypes[domainName];
  }

  return userType(domainName);
}
```

10. Regla de existencia de estructuras y tipos de usuario

La regla semantica de existencia establece que todo tipo definido por usuario que se use como dominio debe existir previamente como defspace. Esto aplica a variables, operadores y estructuras tipadas.

Uso valido:

```
defspace Person end
defspace Group : Person end
```

Uso invalido:

```
defspace Group : Person end
```

// Person no fue declarado como defspace.

El checker registra estos usos con TypePal. Si el espacio requerido no existe, TypePal reporta un error de espacio indefinido.

```
void useDomainIfUserDefined(Domain d, Collector c) {
  if (domainToType(d) is userType) {
    c.use(d, {spaceId()});
  }
}
```

11. Integracion en Main.rsc y Plugin.rsc

11.1 Main.rsc

Main.rsc ejecuta el flujo completo desde la terminal de Rascal. La ruta por defecto usa cwd:/// para ejecutar el programa desde la raiz del proyecto, donde se encuentra la carpeta instance/.

```
int main() {
    return main(|cwd:///instance/spec_types.vl|);
}
```

Si el programa contiene errores semanticos, Main.rsc imprime los errores y retorna 1. Si no hay errores, genera el texto VeriLang y retorna 0.

11.2 Plugin.rsc

Plugin.rsc registra el lenguaje en el entorno, agrega un lente de generacion y ejecuta el checker antes de escribir archivos de salida. Si se encuentran errores, se genera un reporte en instance/output/generator_errors.txt y no se produce codigo generado.

12. Casos de prueba

Archivo	Tipo	Proposito
instance/spec1.vl	Valido	Programa general con espacios, variables, operadores, reglas, atributos, cuantificadores y expresiones.
instance/spec2.vl	Valido	Prueba variables, union, igualdad y expresiones basicas.
instance/spec3.vl	Valido	Prueba una regla unaria con negation.
instance/spec_types.vl	Valido	Prueba tipos primitivos, tipos de usuario, estructura Group : Person y cuantificador sobre Group.
instance/spec_typed_space_good.vl	Valido	Prueba una estructura tipada Set : Element.
instance/spec_typed_space_quantifier_good.vl	Valido	Prueba que el cuantificador sobre Set asigne Element a la variable ligada.
instance/errors/spec_type_error.vl	Invalido	Detecta comparacion entre tipos incompatibles.
instance/errors/spec_unknown_space_error.vl	Invalido	Detecta uso de Person en una estructura tipada sin declararlo como defspace.
instance/errors/spec_logic_error.vl	Invalido	Detecta uso incorrecto de operadores logicos.
instance/errors/spec_operator_error.vl	Invalido	Detecta operador infijo usado con argumentos incorrectos.
instance/errors/spec_rule_error.vl	Invalido	Detecta argumentos incorrectos dentro de defrule.
instance/errors/spec_typed_space_unknown_element_error.vl	Invalido	Detecta una estructura tipada cuyo tipo de elemento no existe.
instance/errors/spec_typed_space_quantifier_type_error.vl	Invalido	Detecta un error de tipo dentro de un cuantificador sobre una estructura tipada.

12.1 Prueba principal valida

```
import Main;
main();
```

Salida esperada:

Archivo de entrada:
|cwd:///instance/spec_types.vl|

Parser ejecutado correctamente.
Checker ejecutado correctamente.

No se encontraron errores semanticos.
Ejecucion finalizada correctamente.
int: 0

12.2 Prueba de existencia de tipo de usuario

```
main(| cwd:///instance/errors/spec_unknown_space_error.vl |);
```

Salida esperada:

El programa contiene errores semanticos:
error("Undefined space `Person`, ...)

No se genera codigo porque el programa no paso el chequeo semantico.
int: 1

12.3 Prueba de tipo dentro de cuantificador

```
main(| cwd:///instance/errors/spec_typed_space_quantifier_type_error.vl |);
```

Salida esperada:

El programa contiene errores semanticos:
error("El lado izquierdo del operador no coincide con su firma", ...)
int: 1

13. Como ejecutar el proyecto

Abrir el proyecto en VS Code o en el entorno Rascal, verificando que la terminal de Rascal este ubicada en la raiz del proyecto. Luego ejecutar:

```
import AST;
import Syntax;
import Parser;
import Implode;
import Generator;
import Checker;
import Main;
import Plugin;
```

Para correr el flujo principal:

```
import Main;
main();
```

Para ejecutar un archivo especifico:

```
main(| cwd:///instance/spec1.vl |);
main(| cwd:///instance/errors/spec_unknown_space_error.vl |);
```

El archivo META-INF/RASCAL.MF declara las librerias necesarias:

```
Manifest-Version: 0.0.1
Project-Name: rascaldslverilang
Source: src/main/rascal
Require-Libraries: |lib://rascal|,|lib://typepal|
```

14. Fuente de la solución

La solución parte del proyecto propio desarrollado en las iteraciones anteriores de VeriLang. No se usó una solución externa completa como base. Las modificaciones se implementaron sobre los archivos del proyecto, principalmente `Syntax.rsc`, `AST.rsc`, `Generator.rsc`, `Checker.rsc`, `Main.rsc` y `Plugin.rsc`. También se organizaron ejemplos válidos e inválidos dentro de `instance/` para demostrar el comportamiento del lenguaje.

Durante el desarrollo se corrigieron problemas de consistencia entre la gramática concreta, el AST, el generador y el checker. La versión final mantiene alineadas las etiquetas de `Syntax.rsc` y `AST.rsc`, conserva el flujo `parser-checker-generator` y agrega las reglas semánticas necesarias para verificar tipos de usuario y estructuras tipadas.

15. Conclusiones

La versión final de VeriLang implementa un flujo completo de lenguaje en Rascal: lee programas `.vl`, los parsea, construye su representación, valida nombres y tipos con `TypePal`, y genera una salida textual cuando el programa es correcto. El lenguaje soporta tipos primitivos, tipos definidos por usuario, estructuras tipadas, operadores lógicos, operadores infijos personalizados, reglas e invocaciones.

Los casos de prueba incluidos demuestran tanto ejecuciones válidas como errores semánticos esperados. En particular, la solución valida la existencia de tipos usados dentro de estructuras y revisa los tipos que aparecen dentro de cuantificadores sobre estructuras tipadas. Con esto, el proyecto queda documentado como una implementación completa y verificable de VeriLang para el Proyecto 3.